Lecture 7: Systems Thinking

Scaling the Frontier Bottleneck

Problem: Compute (ALUs) is far from inputs/outputs (data).

- Last week: parallelism within a single CPU or GPU.
- This week: parallelism across multiple CPUs or GPUs.

Solution: Orchestrate computation to avoid data transfer bottlenecks.

- Single node, single CPU-GPU: L1 cache / shared memory (fastest).
- Single node, single CPU-GPU: HBM ("fast").
- Single node, multi-GPU: NVLink / NVSwitch (slow).
- Multi-node, multi-GPU: Infiniband / Ethernet (slowest).

Mechanics: How to maximize compute and minimize latency?

- Last week: reduce memory accesses via fusion/tiling.
- This week: reduce communication across GPUs/nodes via replication/sharding.

Question: Why do multi-GPU?

1. Your parameters (optimizer state + gradients + activations) don't fit on a single GPU.
2. You want to use more GPUs (more FLOPs) to train faster.

Distributing the Computation

Collective operations are the conceptual primitives used for distributed programming:

- These are classic in the parallel programming literature from the 1980s.
- *Collective* means that you specify a general communication pattern across many devices.
- This can be better/faster than managing point-to-point communication yourself.

Setup for distributed computation and communication:

- **Rank**: a particular CPU or GPU (e.g., 0, 1, 2, 3).

- **World size**: total number of devices (e.g., 4).

Operations for distributed systems and networks:

- Broadcast, scatter, gather, reduce (foundations).
- All-gather, reduce-scatter, all-reduce (workhorse).
- All-to-all (for MoEs).

Broadcast: copy from rank 0 to all ranks.

```
# Input
rank0 = tensor([0., 1, 2, 3])

# Output
rank0 = tensor([0., 1, 2, 3])
rank1 = tensor([0., 1, 2, 3])
rank2 = tensor([0., 1, 2, 3])
rank3 = tensor([0., 1, 2, 3])
```

Scatter tensor on rank 0 to all ranks.

```
# Input
rank0 = tensor([0., 1, 2, 3])

# Output
rank0 = tensor([0.])
rank1 = tensor([1.])
rank2 = tensor([2.])
rank3 = tensor([3.])
```

Gather pieces from all ranks to rank 0 (opposite of scatter).

```
# Input
rank0 = tensor([0.])
rank1 = tensor([1.])
rank2 = tensor([2.])
rank3 = tensor([3.])

# Output
rank0 = tensor([0., 1, 2, 3])
```

Reduce pieces from all ranks to rank 0, applying some operation (e.g, sum, min, max).

```
# Input
rank0 = tensor([0.])
rank1 = tensor([1.])
rank2 = tensor([2.])
rank3 = tensor([3.])

# Output
rank0 = tensor([6.])  # Sum of all ranks (0 + 1 + 2 + 3)
```

All-gather: perform gather to all ranks, not just rank 0.

```
# Input
rank0 = tensor([0.])
rank1 = tensor([1.])
rank2 = tensor([2.])
rank3 = tensor([3.])

# Output
rank0 = tensor([0., 1, 2, 3])
rank1 = tensor([0., 1, 2, 3])
rank2 = tensor([0., 1, 2, 3])
rank3 = tensor([0., 1, 2, 3])
```

Reduce-scatter: perform reduce on each dimension, scatter results.

```
# Input
rank0 = tensor([0., 1, 2, 3])
rank1 = tensor([1., 2, 3, 4])
rank2 = tensor([2., 3, 4, 5])
rank3 = tensor([3., 4, 5, 6])

# Output
rank0 = tensor([6.])  # Sum along dim 0 (0 + 1 + 2 + 3)
rank1 = tensor([10.]) # Sum along dim 1 (1 + 2 + 3 + 4)
rank2 = tensor([14.]) # Sum along dim 2 (2 + 3 + 4 + 5)
rank3 = tensor([18.]) # Sum along dim 3 (3 + 4 + 5 + 6)
```

All-reduce = reduce-scatter + all-gather.

```
# Input
rank0 = tensor([0., 1, 2, 3])
rank1 = tensor([1., 2, 3, 4])
rank2 = tensor([2., 3, 4, 5])
rank3 = tensor([3., 4, 5, 6])
```

```
    # Output
    rank0 = tensor([6., 10, 14, 18])
    rank1 = tensor([6., 10, 14, 18])
    rank2 = tensor([6., 10, 14, 18])
    rank3 = tensor([6., 10, 14, 18])
```

All-to-all: each rank sends each other rank some tensor (most general).

```
    # Input
    rank0 = tensor([0., 1, 2, 3])
    rank1 = tensor([4., 5, 6, 7])
    rank2 = tensor([8., 9, 10, 11])
    rank3 = tensor([12., 13, 14, 15])

    # Output
    rank0 = tensor([0, 4, 8, 12])
    rank1 = tensor([1, 5, 9, 13])
    rank2 = tensor([2, 6, 10, 14])
    rank3 = tensor([3, 7, 11, 15])
```

Notes:

- Useful for MoEs: each rank has split of data and subset of experts; need to route data to experts.
- For balanced splits, all-to-all looks like transpose.
- Also handles unbalanced splits (but want splits to be as balanced as possible).

Way to remember the terminology:

- Reduce - performs some associative/commutative operation (sum, min, max).
- Scatter is inverse of gather.
- All - means destination is all devices.

```
def setup(rank: int, world_size: int):
    """
    Initializes the distributed environment.
    """
    # Specify where master lives (rank 0), used to coordinate
    os.environ["MASTER_ADDR"] = "localhost"
    os.environ["MASTER_PORT"] = "15623"
```

```
if torch.cuda.is_available():
    dist.init_process_group("nccl",
        rank=rank, world_size=world_size)
else:
    dist.init_process_group("gloo",
        rank=rank, world_size=world_size)
```

Hardware Architecture

Classic (in the home):

- GPUs on same node communicate via a PCI(e) bus (v7.0, 16 lanes $\implies$ 242 GB/s).
- GPUs on different nodes communicate via Ethernet (200 MB/s).

Modern (in the data center):

- 8 GPUs per node, connected by NVLink to an NVSwitch (B200s' NVLink 5.0 gets 1.8 TB/s; HBM was 8 TB/s).
- 256 nodes per pod, connected by Infiniband (PCIe $\rightarrow$ HCA / Infiniband NIC $\rightarrow$ Infiniband cable) (0.05 TB/s).
- N pods per cluster / datacenter, connected by Ethernet (PCIe $\rightarrow$ CPU).

Bypassing the CPU:

- Ethernet requires passing through the CPU (copying data to kernel socket buffer, build TCP packets, copy to NIC ring buffer).
- Remote Direct Memory Access (RDMA) - allows one GPU to directly read/write another GPU's memory without involving the CPU.
- Infiniband supports RDMA, but standard Ethernet does not.

Advancements:

- GB200/GB300 NVL72 - 8 GPUs per tray, 9 trays per rack $\rightarrow$ 72 GPUs in one NVLink domain.
- RDMA over Converged Ethernet (RoCE) - Ethernet bypasses CPU, similar but cheaper/weaker than Infiniband, used by Meta.

NVIDIA Collective Communication Library (NCCL)

NCCL translates collective operations into low-level packets that are sent between GPUs.

- Detects topology of hardware (e.g, number of nodes, switches, NVLink/PCIe).
- Optimizes the path between GPUs.
- Launches GPU kernels to send/receive data.

```
def spawn(func: Callable, world_size: int, *args, **kwargs):
    """
    Launches `world_size` processes that each calls `func` on
    world_size, args, kwargs.
    """
    # Note: assume kwargs are in the same order as what main needs
    if not sys.gettrace():
        # This is the normal code path for multiprocessing
        args = (world_size,) + args + tuple(kwargs.values())
        mp.spawn(func, args=args, nprocs=world_size, join=True)
    else:
        with DisableDistributed():
            args = (0, world_size,) + args + tuple(kwargs.values())
            func(*args)
```

Torch Distributed Operations

PyTorch distributed library (**torch.distributed**).

- Provides clean interface for collective operations (e.g., all_ gather_ into_ tensor).
- Supports multiple backends for different hardware: gloo (CPU), nccl (GPU).
- Also supports higher-level algorithms (e.g, FullyShardedDataParallel) [not used in this course].

```
def collective_operations_main(rank: int, world_size: int):
    """This function is running asynchronously
       for each process (rank = 0, ..., world_size - 1)."""
    setup(rank, world_size)

    ### All-reduce (dist = torch.distributed)
    dist.barrier()
    data = tensor([0., 1, 2, 3], device=cuda(rank)) + rank
    print(f"Rank {rank} [before all-reduce]: {data}", flush=True)
    dist.all_reduce(tensor=data,
```

```
                    op=dist.ReduceOp.SUM, async_op=False)
    print(f"Rank {rank} [after all-reduce]: {data}", flush=True)

    ### Reduce-scatter
    dist.barrier()
    input = torch.arange(world_size,
                         dtype=torch.float32,
                         device=cuda(rank)) + rank
    output = torch.empty(1, device=cuda(rank))
    print(f"Rank {rank} [before reduce-scatter]: input = {input},
          output = {output}", flush=True)
    dist.reduce_scatter_tensor(output=output,
                               input=input,
                               op=dist.ReduceOp.SUM,
                               async_op=False)
    print(f"Rank {rank} [after reduce-scatter]: input = {input},
          output = {output}", flush=True)

    ### All-gather
    dist.barrier()
    input = output
    output = torch.empty(world_size, device=cuda(rank))
    print(f"Rank {rank} [before all-gather]: input = {input},
          output = {output}", flush=True)
    dist.all_gather_into_tensor(output_tensor=output,
                                input_tensor=input, async_op=False)
    print(f"Rank {rank} [after all-gather]: input = {input},
          output = {output}", flush=True)
    cleanup()
```

Let's walk through some examples.

```
def all_reduce(rank: int, world_size: int, num_elements: int):
    setup(rank, world_size)

    # Create tensor
    data = torch.randn(num_elements, device=cuda(rank))

    # Warmup
    dist.all_reduce(tensor=data,
                    op=dist.ReduceOp.SUM, async_op=False)
    torch.cuda.synchronize()
    dist.barrier()
```

```
    # Perform all-reduce
    start_time = time.time()
    dist.all_reduce(tensor=data,
                    op=dist.ReduceOp.SUM,
                    async_op=False)
    torch.cuda.synchronize()
    dist.barrier()
    end_time = time.time()
    duration = end_time - start_time

    # Measure the effective bandwidth
    dist.barrier()
    size_bytes = data.element_size() * data.numel()
    sent_bytes = size_bytes * 2 * (world_size - 1)
    total_duration = world_size * duration
    bandwidth = sent_bytes / total_duration

    cleanup()
```

Notes:

- Effective bandwidth = $2\times$ size_bytes / total_duration.
- Independent of world_size.
- Independent of topology (ring or tree).

```
def reduce_scatter(rank: int, world_size: int, num_elements: int):
    setup(rank, world_size)

    # Create input and outputs
    input = torch.randn(world_size, num_elements, device=cuda(rank))
    output = torch.empty(num_elements, device=cuda(rank))

    # Warmup
    dist.reduce_scatter_tensor(
        input=input,output=output,
        op=dist.ReduceOp.SUM, async_op=False)
    torch.cuda.synchronize()
    dist.barrier()
    # Perform reduce-scatter
    start_time = time.time()
    dist.reduce_scatter_tensor(
        output=output, input=input,
```

```
        op=dist.ReduceOp.SUM, async_op=False)
    torch.cuda.synchronize()
    dist.barrier()
    end_time = time.time()
    duration = end_time - start_time

    # Measure the effective bandwidth
    dist.barrier()
    data_bytes = input.element_size() * input.numel()
    sent_bytes = data_bytes * (world_size - 1)
    total_duration = world_size * duration
    bandwidth = sent_bytes / total_duration
    cleanup()
```

Notes:

- all-reduce = reduce-scatter + all-gather.
- all-reduce moves $2\times$ the data in $2\times$ the time compared to reduce-scatter.

```
def cleanup():
    """Cleans up the distributed environment."""
    torch.distributed.destroy_process_group()
```

Distributing the Training

Walk through bare-bones implementations of each strategy on deep MLPs. Recall that MLPs are the compute bottleneck in Transformers, so this is representative.

What's missing?

- Communication/computation overlap.
- More general models (with attention, etc).
- Other forms of parallelism (e.g., sequence parallelism, expert parallelism, combinations).
- Jax/TPUs: just define the model, the sharding strategy, and the Jax compiler handles the rest.
- But we're doing PyTorch so you can see how one builds up from the primitives.

Data Parallelism

Sharding strategy: each rank gets a slice of the data.

- Losses are different across ranks (computed on local data).
- Gradients are all-reduced to be the same across ranks.
- Therefore, parameters remain the same across ranks.

```
def data_parallelism_main(rank: int,
    world_size: int, data: tensor,
    num_layers: int, num_steps: int):
    setup(rank, world_size)

    # Get the slice of data for this rank
    # --- B0 ---
    # --- B1 ---
    # --- B2 ---
    # --- B3 ---
    batch_size = data.size(0)
    num_dim = data.size(1)
    local_batch_size = int_divide(batch_size, world_size)
    start_index = rank * local_batch_size
    end_index = start_index + local_batch_size
    data = data[start_index:end_index].to(cuda(rank))

    # Create MLP parameters params[0], ..., params[num_layers - 1]
    params = [
        get_init_params(num_dim, num_dim, rank)
        for layer in range(num_layers)]
    optimizer = torch.optim.AdamW(params, lr=1e-3)
    for step in range(num_steps):
        # Forward pass
        x = data
        for param in params:
            x = x @ param
            x = torch.nn.functional.gelu(x)
        loss = x.square().mean()
        # Backward pass
        loss.backward()
        # Sync gradients across workers
        for param in params:
            dist.all_reduce(tensor=param.grad,
```

```
                                op=dist.ReduceOp.AVG, async_op=False)
        # Update parameters
        optimizer.step()
    cleanup()
```

Tensor Parallelism

Sharding strategy: each rank gets part of each layer, transfer all data/activations.

```
def tensor_parallelism_main(rank: int, world_size: int,
    data: tensor, num_layers: int):
    setup(rank, world_size)
    data = data.to(cuda(rank))
    batch_size = data.size(0)
    num_dim = data.size(1)
    local_num_dim = int_divide(num_dim, world_size)

    # Create model
    #  |  |  |  |
    # W0 W1 W2 W3
    #  |  |  |  |
    params = [
        get_init_params(num_dim, local_num_dim, rank)
        for layer in range(num_layers)]

    # Forward pass
    x = data
    for layer in range(num_layers):
        # Compute activations (batch_size x local_num_dim)
        x = x @ params[layer]
        x = torch.nn.functional.gelu(x)
        # Allocate memory for activations
        activations = [
            torch.empty(batch_size, local_num_dim,
            device=cuda_if_available(rank))
            for _ in range(world_size)]
        # Send activations via all gather
        dist.all_gather(
            tensor_list=activations,
            tensor=x, async_op=False)
        # Concatenate them to get batch_size x num_dim
        x = torch.cat(activations, dim=1)
```

```
    # Backward pass: homework exercise
    cleanup()
```

Pipeline Parallelism

Sharding strategy: each rank gets subset of layers, transfer all data/activations.

```
def pipeline_parallelism_main(rank: int, world_size: int,
    data: tensor, num_layers: int, num_micro_batches: int):
    setup(rank, world_size)

    # Use all the data
    data = data.to(cuda_if_available(rank))
    batch_size = data.size(0)
    num_dim = data.size(1)

    # Split up layers
    local_num_layers = int_divide(num_layers, world_size)
    # Each rank gets a subset of layers
    local_params = [
        get_init_params(num_dim, num_dim, rank)
        for layer in range(local_num_layers)]

    # Forward pass
    # Break up into micro batches to minimize the bubble
    micro_batch_size = int_divide(batch_size, num_micro_batches)
    if rank == 0:
        # The data
        micro_batches = data.chunk(chunks=num_micro_batches, dim=0)
    else:
        # Allocate memory for activations
        micro_batches = [
            torch.empty(micro_batch_size, num_dim,
            device=cuda_if_available(rank))
            for _ in range(num_micro_batches)]
    for x in micro_batches:
        # Get activations from previous rank
        if rank - 1 >= 0:
            dist.recv(tensor=x, src=rank - 1)
        # Compute layers assigned to this rank
        for param in local_params:
            x = x @ param
            x = F.gelu(x)
```

```
        # Send to the next rank
        if rank + 1 < world_size:
            dist.send(tensor=x, dst=rank + 1)

    # Backward pass: homework exercise
    cleanup()
```

Collective Parallelism

- Many ways to parallelize: data (batch), tensor/expert (width), pipeline (depth), sequence (length).
- Data parallelism: DDP (all-reduce), FSDP/ZeRO (all-gather + reduce-scatter).
- Tensor parallelism: requires very fast interconnects (e.g., NVLink).
- Pipeline parallelism: can work with slow interconnects, but need to work to reduce pipeline bubbles.
- Can re-compute or store in memory or store in another GPUs memory and communicate.
- Hardware is getting faster, but will always want bigger models, so will have this hierarchical structure.